

Project Executable Files

Date	30 September 2026
Team ID	SWTID-2026-2056
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	3 Marks

This section requires submission of all executable and deployable configuration artifacts for the project **Implement Client Script & UI Policy (Incident)**. Ensure all files, scripts, and configuration records are properly organized and documented so that the evaluator can verify, deploy, or reproduce the solution using the provided materials and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source / configuration artifacts (Update Set XML export or screenshots of all UI Policies & Client Scripts)	Yes
2	README / Setup Guide (instructions to configure and test the solution on a ServiceNow instance)	Yes
3	requirements.txt / package.json / dependency file	NA
4	Database schema / seed files (if applicable)	NA
5	Environment configuration file (.env.example or similar)	NA
6	Deployed application URL (ServiceNow instance URL if hosted/demo instance provided)	Yes
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results (test evidence / screenshots of validation scenarios)	Yes
10	Demo video or walkthrough (if required)	Yes / NA

Step 2: File / Folder Structure

Provide an overview of the project artifacts and configuration structure below:

```
/project-root
├── update-sets/
│   └── Client_Script_UI_Policy_Incident.xml # ServiceNow Update Set export
├── scripts/
│   ├── onChange_AutoSetUrgency.js # Task 3 - onChange Client Script
│   ├── onSubmit_PreventSave.js # Task 4 - onSubmit Client Script
│   └── onCellEdit_PreventStateChange.js # Task 5 - onCellEdit Client Script
├── ui-policies/
│   └── High_Impact_Control.txt # UI Policy definition + Action details
├── docs/
│   ├── README.md # Setup & run instructions
│   └── testing-evidence/ # Screenshots of test scenarios
│       ├── test_mandatory_enforcement.png
│       ├── test_successful_save.png
│       ├── test_reverse_condition.png
│       ├── test_list_edit_blocking.png
│       └── test_form_based_update.png
└── demo/
    └── walkthrough_video_link.txt # Link to demo video (if any)
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	[Your ServiceNow Instance URL, e.g., https://devXXXXX.service-now.com]
Login Credentials (Demo)	[Provide demo username and password for evaluator access]
Platform / Hosting Provider	ServiceNow (Cloud / Personal Developer Instance)
Repository Link	[GitHub / GitLab link if configuration files are version-controlled]
Demo Video Link	[YouTube / Drive / Loom link to walkthrough video, if required]
Update Set Name	Client Script & UI Policy – Incident

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to configure and test the solution on a ServiceNow instance:

Pre-requisites:

- Active ServiceNow instance (Personal Developer Instance recommended) with admin or configuration access
- Roles: admin, itil, or equivalent access to System UI → UI Policies and Client Scripts
- Browser: Latest Chrome / Edge / Firefox

- Optional: ServiceNow Update Set XML file if importing configurations

Step 1: Create UI Policy – High Impact Control

- Log in to the ServiceNow instance with administrative access.
- Navigate to System UI → UI Policies and click New.
- Fill in the details:
 - Name: High Impact Control
 - Table: Incident [incident]
 - Active: true
 - Reverse if false: true
- Condition: Impact is 1 – High
- Click Submit.

Step 2: Create UI Policy Action – Urgency (Read-only)

- Open the UI Policy High Impact Control.
- In the related list UI Policy Actions, click New.
- Configure:
 - Field name: Urgency
 - Read only: true
 - Visible: leave default
- Click Submit.

Step 3: Create onChange Client Script – Auto set Urgency

- Navigate to System UI → Client Scripts → New.
- Configure:
 - Name: Auto set urgency for high impact
 - Table: Incident
 - Type: onChange
 - Field name: Impact
 - Active: true
- Paste the following script and click Submit:

```
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue == '') {
        return;
    }
    if (newValue == '1') {
        g_form.setValue('urgency', '1');
        g_form.addInfoMessage('Urgency set to High for High impact incident.');
```

Step 4: Create onSubmit Client Script – Prevent Save

- Navigate to System UI → Client Scripts → New.
- Configure:
 - Name: Prevent save if Assigned To missing
 - Table: Incident
 - Type: onSubmit
 - Active: true
- Paste the following script and click Submit:

```
function onSubmit() {
    if (g_form.getValue('impact') == '1' &&
        g_form.getValue('assigned_to') == '') {
        g_form.showErrorMessage(
            'assigned_to',
            'Assigned To is mandatory for High impact incidents.'
        );
        return false;
    }
    return true;
}
```

Step 5: Create onCellEdit Client Script – Prevent State Change

- Navigate to System UI → Client Scripts → New.
- Configure:
 - Name: Prevent state change via list edit
 - Table: Incident
 - Type: onCellEdit

- Field name: State
- Active: true
- Paste the following script and click Submit:

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    alert('State cannot be updated using list editing. Please open the Incident.');
```

```
    callback(false);
}
```

Step 6: Testing the Configuration

After creating all UI Policies and Client Scripts, perform the following verification steps:

Activity 1 – Test Mandatory Enforcement

- Go to Incident → Create New.
- Set Impact = High. Observe that Urgency becomes read-only and is auto-set to High.
- Leave Assigned To empty and click Submit.
- Verify: Save is blocked and error message appears on Assigned To field.

Activity 2 – Test Successful Save

- Fill in the Assigned To field with a valid user.
- Click Submit. Record should save successfully.
- Confirm Urgency is High and read-only while Impact remains High.

Activity 3 – Reverse Condition Test

- Open an existing High-impact Incident.
- Change Impact from High to Medium.
- Verify: Urgency becomes editable again (Reverse if false works).
- Save the record successfully.

Activity 4 – Test List Edit Blocking

- Navigate to Incident → All.
- Double-click the State field on any row to attempt list editing.
- Verify: Alert appears and the change is blocked.

Activity 5 – Test Form-Based Update

- Open the same Incident in form view.
- Change the State field and click Update.
- Verify: State change is allowed via the form (only list edit is blocked).

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	onCellEdit script blocks all list edits on State; legitimate bulk updates require form or workflow.	Documented behavior. Use form view or dedicated workflow for bulk state changes.
2	UI Policy read-only on Urgency applies only when Impact = High; other priority combinations are not controlled by this policy.	By design for the micro-project scope. Additional policies can be added if needed.
3	Client Scripts run only in the browser; they do not enforce rules for integrations or API-based record creation.	For full enforcement, pair with server-side Business Rules or Data Policies.

Conclusion

The **Implement Client Script & UI Policy (Incident)** micro project demonstrates how ServiceNow client-side controls can enforce data integrity on Incident records. By combining a UI Policy (with Reverse if false) and three Client Scripts (onChange, onSubmit, onCellEdit), the solution dynamically makes Urgency read-only, auto-populates Urgency for High Impact, prevents save when Assigned To is missing, and blocks unauthorized list edits on State. The configuration is lightweight, easy to deploy via Update Set, and provides clear evidence of improved form usability and data quality.

All configuration steps, scripts, and test scenarios are documented above so that an evaluator can reproduce and verify the solution on any standard ServiceNow instance.